



US012417556B2

(12) **United States Patent**
Tian et al.

(10) **Patent No.: US 12,417,556 B2**

(45) **Date of Patent: Sep. 16, 2025**

(54) **TEMPORAL PREDICTION BASED VERTEX POSITION COMPRESSION**

(56) **References Cited**

U.S. PATENT DOCUMENTS

(71) Applicant: **Tencent America LLC**, Palo Alto, CA (US)

2018/0189982 A1* 7/2018 Laroche G06T 9/40
2021/0174551 A1* 6/2021 Graziosi G06T 15/08

(Continued)

(72) Inventors: **Jun Tian**, Belle Mead, NJ (US); **Chao Huang**, Palo Alto, CA (US); **Xiang Zhang**, Sunnyvale, CA (US); **Xiaozhong Xu**, State College, PA (US); **Shan Liu**, San Jose, CA (US)

FOREIGN PATENT DOCUMENTS

JP 2008-516318 A 5/2008

OTHER PUBLICATIONS

Stefan Gumhold and Wolfgang StraBer. 1998. Real time compression of triangle mesh connectivity. In Proceedings of the 25th annual conference on Computer graphics and interactive techniques (SIGGRAPH '98). ACM, New York, NY, USA, 133-140. <https://doi.org/10.1145/280814.28> (Year: 1998).*

(Continued)

(21) Appl. No.: **18/127,487**

(22) Filed: **Mar. 28, 2023**

Primary Examiner — Kee M Tung

Assistant Examiner — Nauman U Ahmad

(74) *Attorney, Agent, or Firm* — ArentFox Schiff LLP

(65) **Prior Publication Data**

US 2023/0388543 A1 Nov. 30, 2023

Related U.S. Application Data

(60) Provisional application No. 63/345,824, filed on May 25, 2022.

(51) **Int. Cl.**
H04N 19/46 (2014.01)
G06T 9/00 (2006.01)
H04N 19/172 (2014.01)

(52) **U.S. Cl.**
CPC **G06T 9/001** (2013.01); **H04N 19/172** (2014.11); **H04N 19/46** (2014.11)

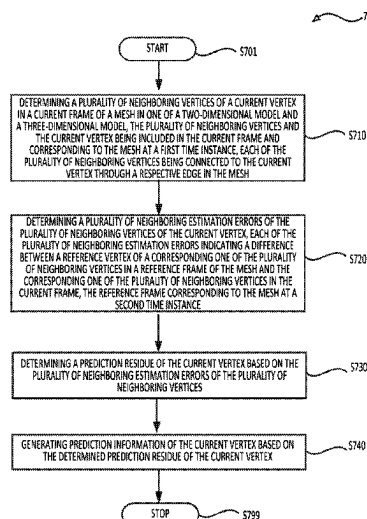
(58) **Field of Classification Search**
CPC H04N 19/172; H04N 19/46; H04N 19/105; H04N 19/154; H04N 19/597;

(Continued)

(57) **ABSTRACT**

A plurality of neighboring vertices of a current vertex in a current frame of a mesh is determined. The current frame corresponds to the mesh at a first time instance. Each of the plurality of neighboring vertices is connected to the current vertex through a respective edge in the mesh. A plurality of neighboring estimation errors of the plurality of neighboring vertices is determined. Each of the plurality of neighboring estimation errors indicates a difference between a reference vertex of a corresponding one of the plurality of neighboring vertices in a reference frame of the mesh and the corresponding one of the plurality of neighboring vertices in the current frame. The reference frame corresponds to the mesh at a second time instance. A prediction residue of the current vertex is determined based on the plurality of neighboring estimation errors. Prediction information of the current vertex is generated based on the determined prediction residue.

20 Claims, 9 Drawing Sheets



(58) **Field of Classification Search**

CPC H04N 19/54; H04N 19/14; H04N 19/70;
G06T 17/205; G06T 9/001

See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

2021/0217203	A1 *	7/2021	Kim	G06T 9/001
2022/0108483	A1 *	4/2022	Graziosi	G06T 17/20
2023/0410374	A1 *	12/2023	Ahn	G06T 9/001
2024/0185467	A1 *	6/2024	Lee	G06T 9/001

OTHER PUBLICATIONS

Rossignac, Jarek & Safonova, Alla & Szymczak, Andrzej. (2001). 3D Compression Made Simple: Edgebreaker on a Corner-Table. (Year: 2001).*

Frédéric Payan, Marc Antonini. Wavelet-based Compression of 3D Mesh Sequences. (Year: 2005).*

E. Faramarzi, R. Joshi and M. Budagavi, "Mesh Coding Extensions to MPEG-I V-PCC," 2020 IEEE 22nd International Workshop on

Multimedia Signal Processing (MMSP), Tampere, Finland, 2020, pp. 1-5, doi: 10.1109/MMSP48831.2020.9287057 (Year: 2020).*

Lawrence Ibarria and Jarek Rossignac. 2003. Dynapack: space-time compression of the 3D animations of triangle meshes with fixed connectivity. In Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation (SCA '03). Eurographics Association, Goslar, DEU. (Year: 2003).*

Touma, Costa, and Craig Gotsman. "Triangle mesh compression." Proceedings-Graphics Interface. Canadian Information Processing Society, 1998, pp. 1-9.

Maglo, Adrien, et al. "3d mesh compression: Survey, comparisons, and emerging trends." ACM Computing Surveys (CSUR) 47.3 (2015): 1-41.

A. S. Jarek Rossignac and A. Szymczak. "3d compression made simple: Edgebreaker on a corner table". In Proceedings of Shape Modeling International Conference, Genoa, Italy, 2001, pp. 1-7.

Office Action received for Japanese Patent Application No. 2024-515865, mailed on Feb. 12, 2025, 12 pages (7 pages of English Translation and 5 pages of Original Document).

Yang et al., "Compression of 3D Triangle Mesh Sequences", 2001 IEEE Fourth Workshop on Multimedia Signal Processing, 2001, pp. 181-186.

* cited by examiner

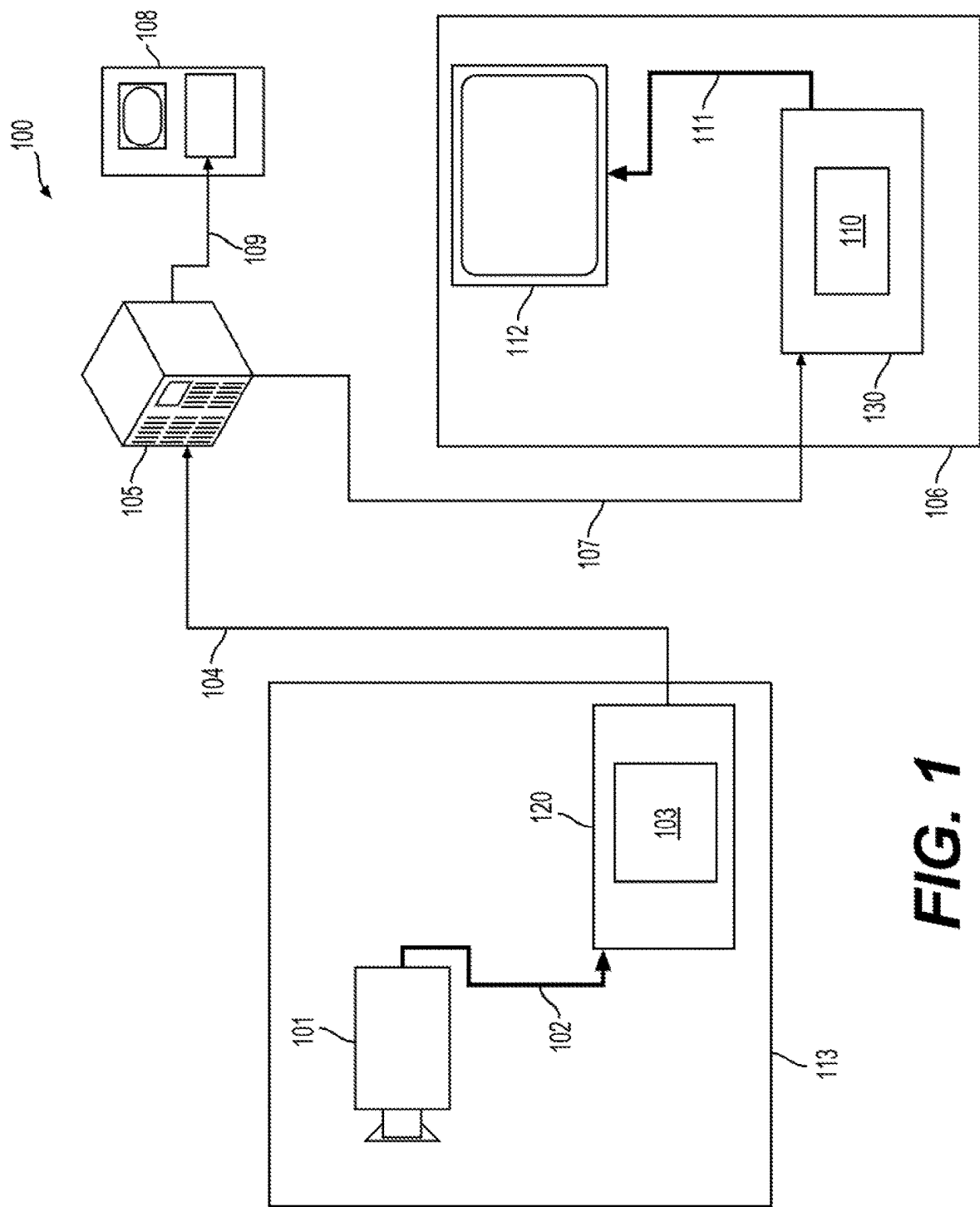
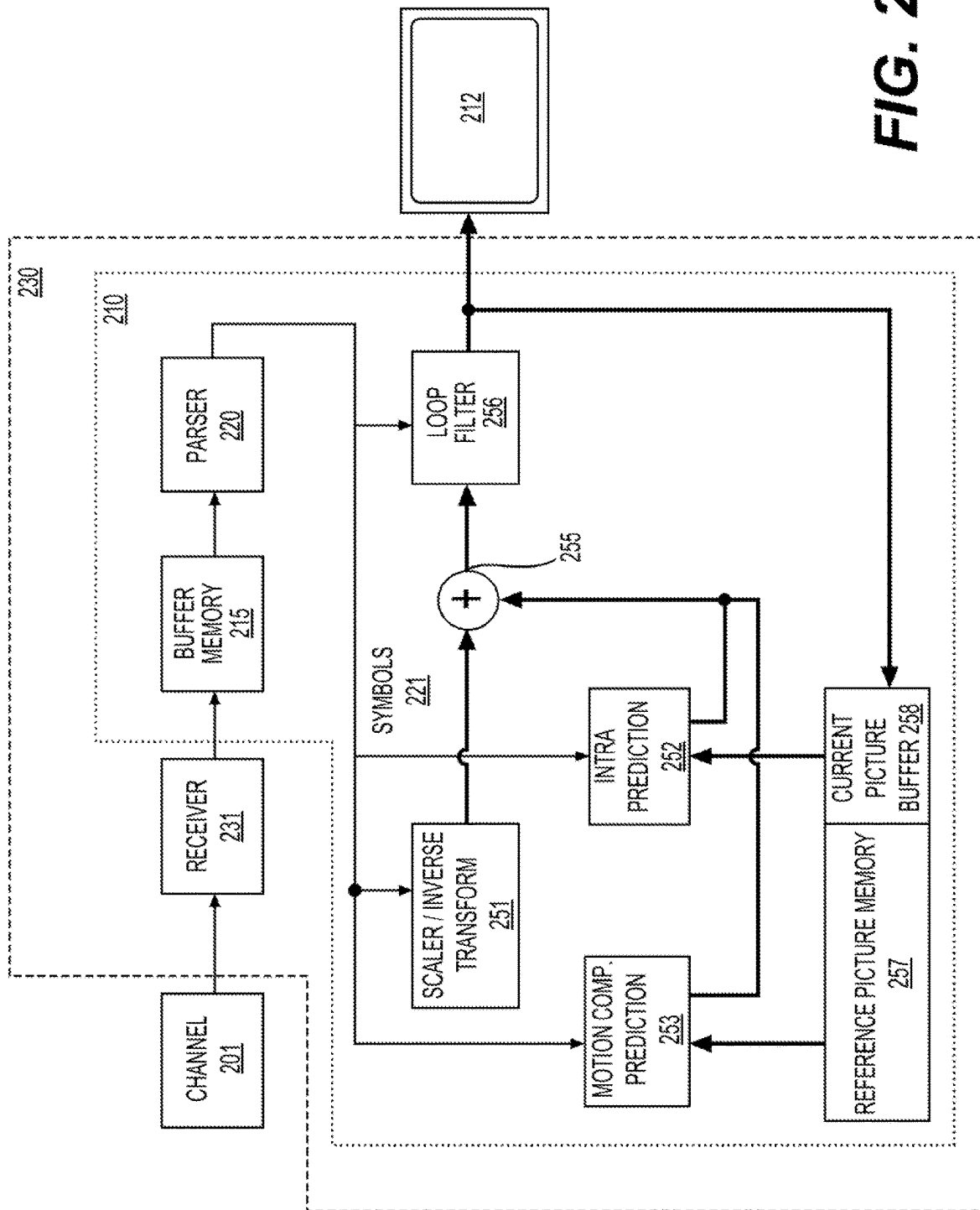
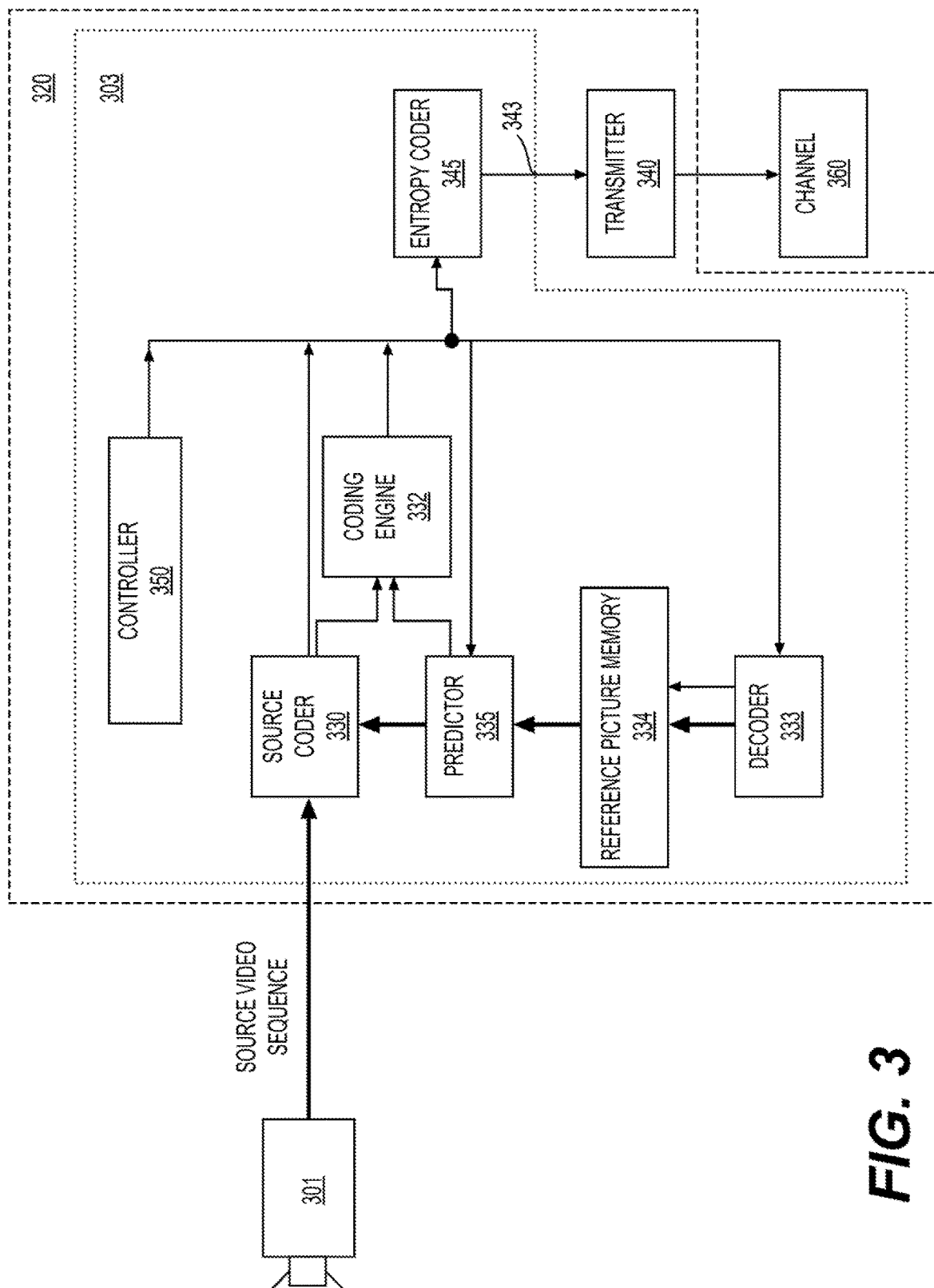


FIG. 1





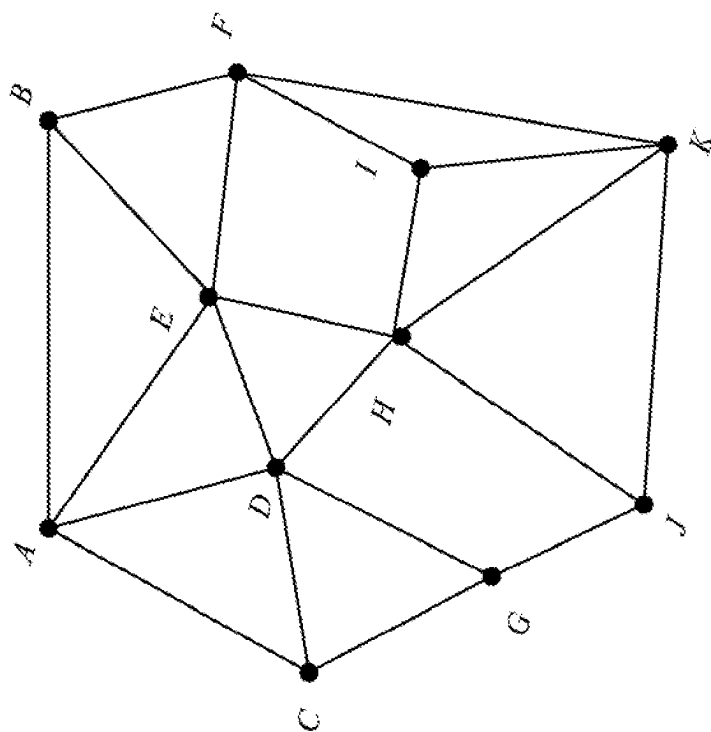
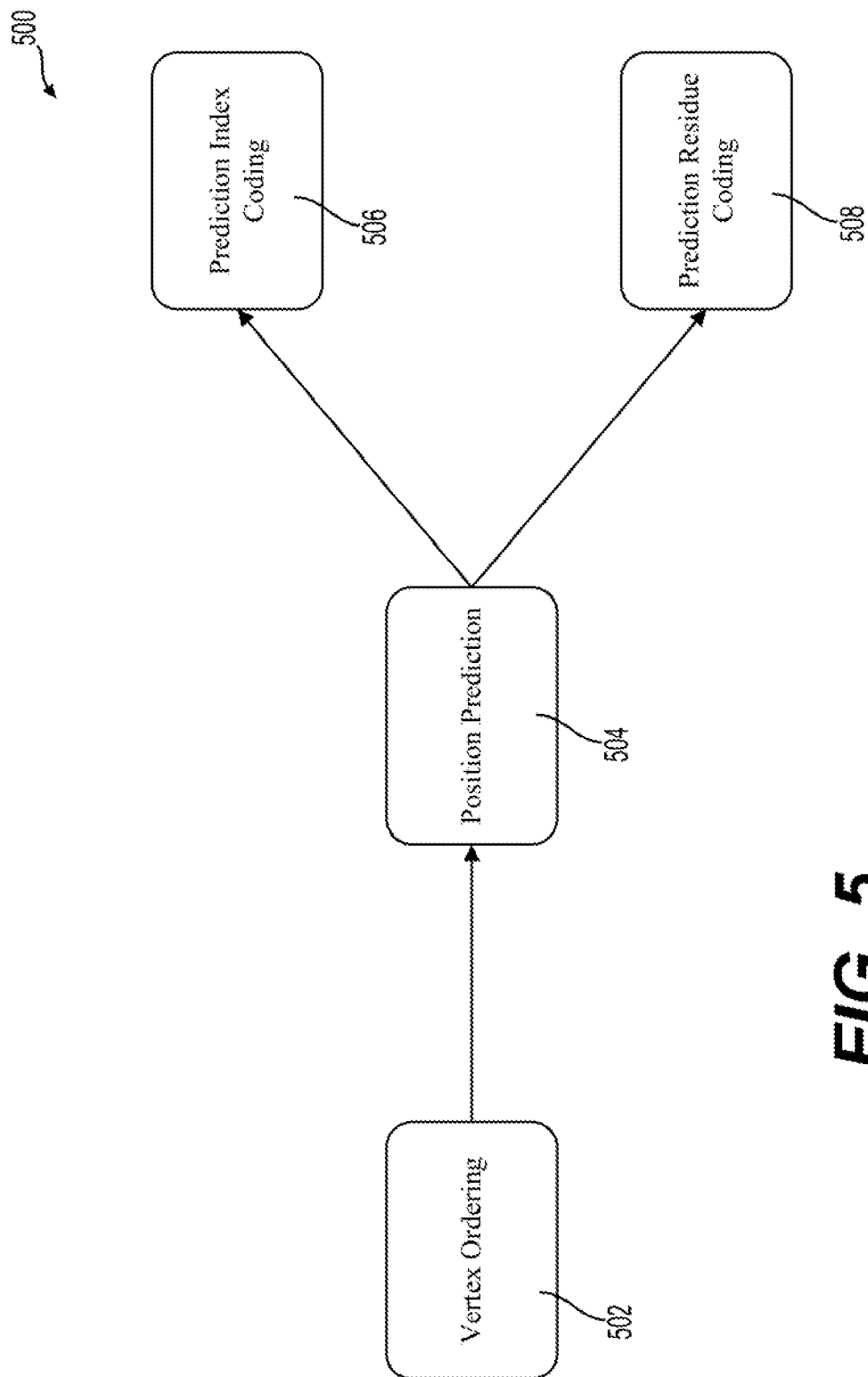


FIG. 4

**FIG. 5**

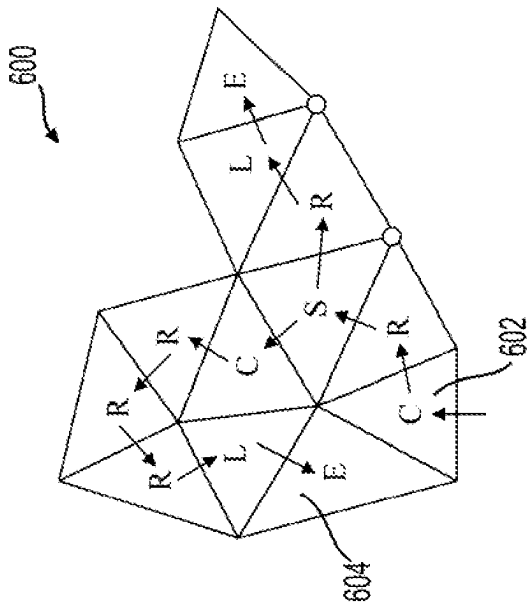


FIG. 6B

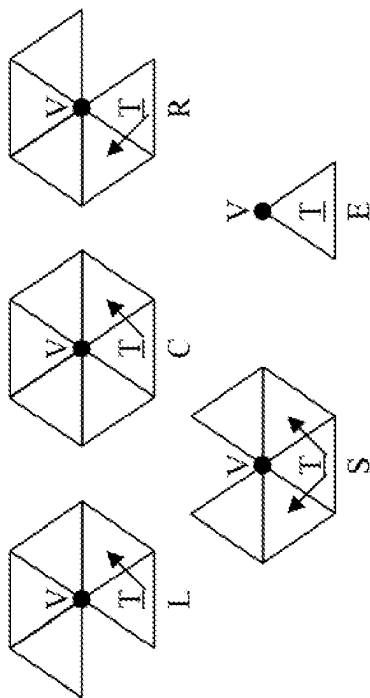
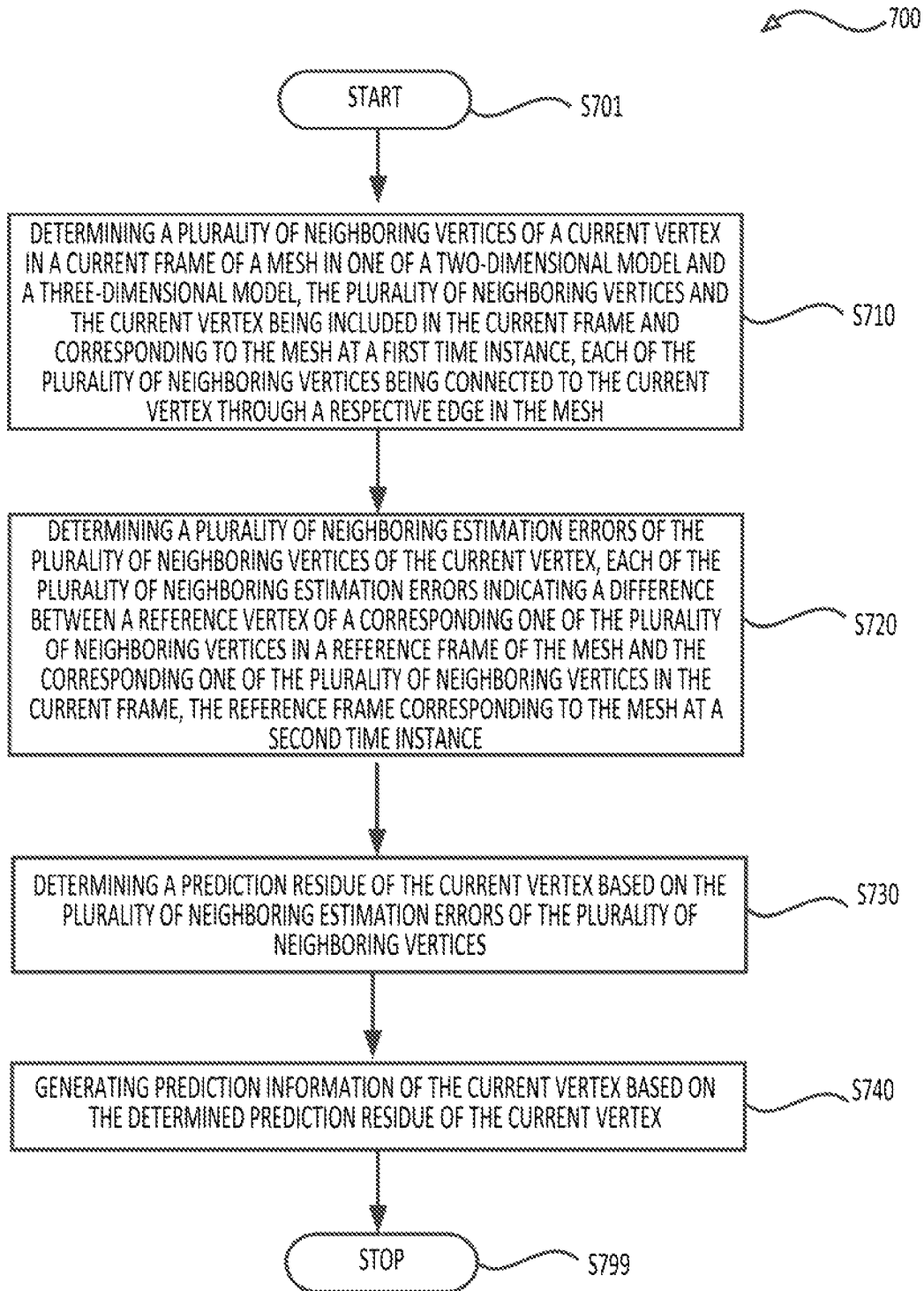
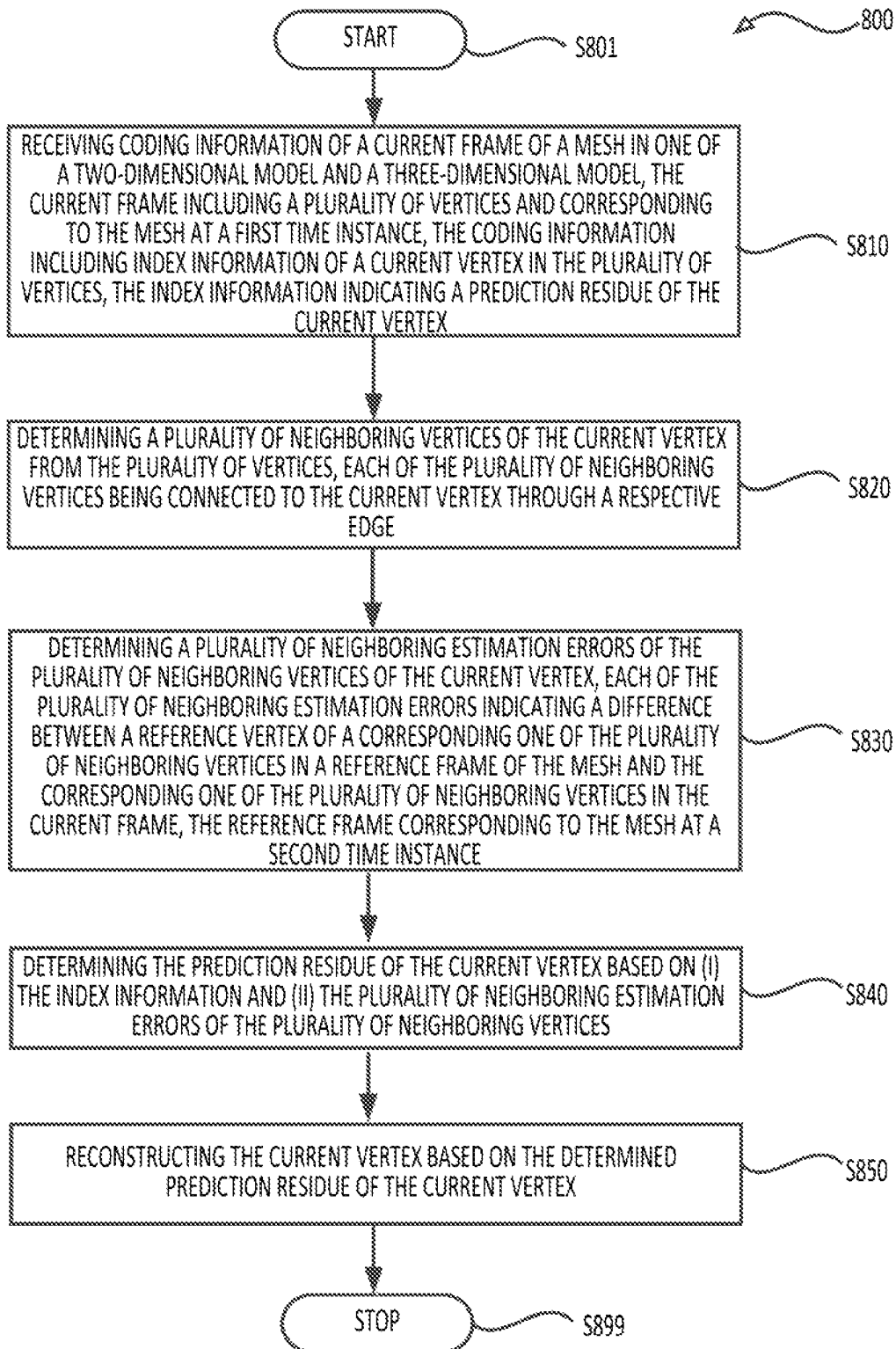


FIG. 6A

**FIG. 7**

**FIG. 8**

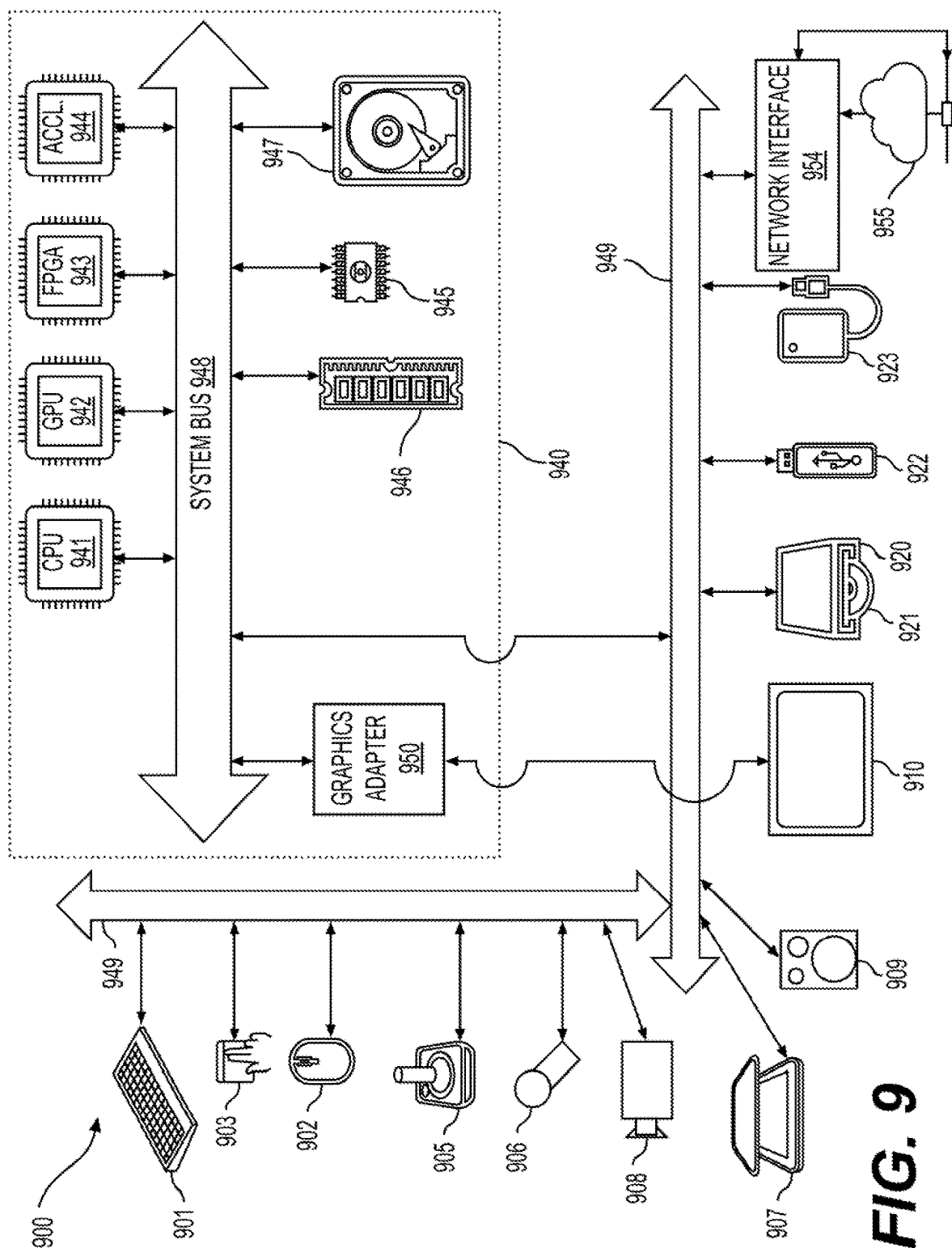


FIG. 9

1

TEMPORAL PREDICTION BASED VERTEX POSITION COMPRESSION

INCORPORATION BY REFERENCE

The present application claims the benefit of priority to U.S. Provisional Application No. 63/345,824, "Temporal Prediction based Vertex Position Compression" filed on May 25, 2022, which is incorporated by reference herein in its entirety.

TECHNICAL FIELD

The present disclosure includes embodiments related to mesh processing.

BACKGROUND

The background description provided herein is for the purpose of generally presenting the context of the disclosure. Work of the presently named inventors, to the extent the work is described in this background section, as well as aspects of the description that may not otherwise qualify as prior art at the time of filing, are neither expressly nor impliedly admitted as prior art against the present disclosure.

Advances in three-dimensional (3D) capture, modeling, and rendering have promoted ubiquitous presence of 3D content across various platforms and devices. Nowadays, it is possible to capture a baby's first step in one continent and allow grandparents of the baby to see (and in some cases interact) and enjoy a full immersive experience with the child in another continent. In order to achieve such realism, models are becoming ever more sophisticated, and a significant amount of data is linked to the creation and consumption of those models. 3D meshes are widely used to represent such immersive contents.

SUMMARY

Aspects of the disclosure provide methods and apparatuses for mesh processing. In some examples, an apparatus for mesh processing includes processing circuitry.

According to an aspect of the disclosure, a method of mesh processing performed in a video encoder is provided. In the method, a plurality of neighboring vertices of a current vertex is determined in a current frame of a mesh in one of a two-dimensional model and a three-dimensional model. The current vertex and the plurality of neighboring vertices are included in the current frame and corresponds to the mesh at a first time instance. Each of the plurality of neighboring vertices is connected to the current vertex through a respective edge in the mesh. A plurality of neighboring estimation errors of the plurality of neighboring vertices of the current vertex is determined. Each of the plurality of neighboring estimation errors indicates a difference between a reference vertex of a corresponding one of the plurality of neighboring vertices in a reference frame of the mesh and the corresponding one of the plurality of neighboring vertices in the current frame. The reference frame corresponds to the mesh at a second time instance. A prediction residue of the current vertex is determined based on the plurality of neighboring estimation errors of the plurality of neighboring vertices. Prediction information of the current vertex is generated based on the determined prediction residue of the current vertex.

2

In some embodiments, the reference vertex of the corresponding one of the plurality of neighboring vertices is positioned at a same relative position in the reference frame as the current vertex in the current frame, where the reference frame and the current frame are generated at different time instances.

In an example, an average neighboring estimation error of the plurality of neighboring estimation errors is determined. The prediction residue of the current vertex is determined based on the average neighboring estimation error.

In an example, to determine the prediction residue of the current vertex, an estimation error of the current vertex is determined based on a difference between a reference vertex of the current vertex and the current vertex. A prediction list of the current vertex is determined. Predictors of the prediction list includes the estimation error of the current vertex, the average neighboring estimation error subsequent to the estimation error of the current vertex, and the plurality of neighboring estimation errors subsequent to the average neighboring estimation error. A respective prediction index is associated with each of the predictors in the prediction list.

In some embodiments, the plurality of neighboring vertices is ordered based on an Edgebreaker algorithm in which the plurality of neighboring vertices is traversed in a spiraling triangle-spanning-tree order.

In an example, to determine the prediction residue of the current vertex, a difference between the average neighboring estimation error and the estimation error of the current vertex is determined. A difference between each of the plurality of neighboring estimation errors and the estimation error of the current vertex is also determined. A neighboring estimation error from the average neighboring estimation error and the plurality of neighboring estimation errors is selected. The selected neighboring estimation error has a minimum difference. The prediction residue of the current vertex is determined as one of (i) the estimation error of the current vertex and (ii) the selected neighboring estimation error.

In an example, to determine the prediction residue of the current vertex, a prediction error is determined based on one of the plurality of neighboring vertices of the current vertex according to an inter frame prediction. The prediction error is compared with each of the predictors in the prediction list. The prediction residue is determined as the prediction error in response to the prediction error being smaller than the predictors in the prediction list.

In an example, to determine the prediction residue of the current vertex, the prediction residue is determined as a mixed estimation error. The mixed estimation error includes one of (i) an average of the selected neighboring estimation error and the prediction error and (ii) an average of the estimation error of the current vertex and the prediction error.

In an embodiment, the inter frame prediction further includes one of a delta prediction that is based on a difference between a first prediction vertex and the current vertex, where the first prediction vertex includes the one of the plurality of neighboring vertices of the current vertex. The inter frame prediction can also include a parallelogram prediction in which a second prediction vertex is determined based on a prediction triangle of a plurality of triangles of the mesh. The prediction triangle shares an edge with a triangle of the plurality of triangles that includes the current vertex. The second prediction vertex is opposite to the shared edge, and the second prediction vertex and the prediction triangle form a parallelogram.

In an embodiment, the prediction information of the current vertex further includes a flag. The flag indicates that the prediction residue is one of (i) the estimation error of the current vertex or the selected neighboring estimation error based on a first value of the flag, (ii) the prediction error based on a second value of the flag, and (iii) the mixed estimation error based on a third value of the flag.

In an embodiment, the prediction information of the current vertex further includes index information indicating which one of the predictors in the prediction list is the prediction residue based on the flag being the first value. The prediction information also includes prediction residue information. In an example, based on the flag being the first value, the prediction residue information indicates (i) the estimation error of the current vertex in response to the index information indicating that the prediction residue is the estimation error of the current vertex, and (ii) the selected neighboring estimation error in response to the index information indicating that the prediction residue is the selected neighboring estimation error. Based on the flag being the second value, the prediction residue information indicates a difference between the current vertex and a prediction vertex indicated by the prediction error. Based on the flag being the third value, the prediction residue information indicates a difference between the current vertex and a prediction vertex indicated by the mixed estimation error.

According to another aspect of the disclosure, an apparatus is provided. The apparatus includes processing circuitry. The processing circuitry can be configured to perform any of the described methods for mesh processing.

Aspects of the disclosure also provide a non-transitory computer-readable medium storing instructions which when executed by a computer for video decoding cause the computer to perform any of the described methods for mesh processing.

BRIEF DESCRIPTION OF THE DRAWINGS

Further features, the nature, and various advantages of the disclosed subject matter will be more apparent from the following detailed description and the accompanying drawings in which:

FIG. 1 is a schematic illustration of an exemplary block diagram of a communication system (100).

FIG. 2 is a schematic illustration of an exemplary block diagram of a decoder.

FIG. 3 is a schematic illustration of an exemplary block diagram of an encoder.

FIG. 4 is a schematic illustration of vertices in a mesh according to some embodiments of the disclosure.

FIG. 5 is a schematic illustration of an exemplary vertex position compression according to some embodiments of the disclosure.

FIG. 6A shows an example of five patch configurations of an Edgebreaker algorithm.

FIG. 6B shows an exemplary mesh traversal according to an Edgebreaker algorithm.

FIG. 7 shows a flow chart outlining a process according to some embodiments of the disclosure.

FIG. 8 shows a flow chart outlining another process according to some embodiments of the disclosure.

FIG. 9 is a schematic illustration of a computer system in accordance with an embodiment.

DETAILED DESCRIPTION OF EMBODIMENTS

FIG. 1 shows a block diagram of a video processing system (100) in some examples. The video processing

system (100) is an example of an application for the disclosed subject matter, a video encoder and a video decoder in a streaming environment. The disclosed subject matter can be equally applicable to other image and video enabled applications, including, for example, video conferencing, digital TV, streaming services, storing of compressed video on digital media including CD, DVD, memory stick and the like.

The video processing system (100) includes a capture subsystem (113), that can include a video source (101). The video source (101) can include one or more images captured by a camera and/or generated by a computer. For example, a digital camera can create a stream of video pictures (102) that are uncompressed. In an example, the stream of video pictures (102) includes samples that are taken by the digital camera. The stream of video pictures (102), depicted as a bold line to emphasize a high data volume when compared to encoded video data (104) (or coded video bitstreams), can be processed by an electronic device (120) that includes a video encoder (103) coupled to the video source (101). The video encoder (103) can include hardware, software, or a combination thereof to enable or implement aspects of the disclosed subject matter as described in more detail below. The encoded video data (104) (or encoded video bitstream), depicted as a thin line to emphasize the lower data volume when compared to the stream of video pictures (102), can be stored on a streaming server (105) for future use. One or more streaming client subsystems, such as client subsystems (106) and (108) in FIG. 1 can access the streaming server (105) to retrieve copies (107) and (109) of the encoded video data (104). A client subsystem (106) can include a video decoder (110), for example, in an electronic device (130). The video decoder (110) decodes the incoming copy (107) of the encoded video data and creates an outgoing stream of video pictures (111) that can be rendered on a display (112) (e.g., display screen) or other rendering device (not depicted). In some streaming systems, the encoded video data (104), (107), and (109) (e.g., video bitstreams) can be encoded according to certain video coding/compression standards. Examples of those standards include ITU-T Recommendation H.265. In an example, a video coding standard under development is informally known as Versatile Video Coding (VVC). The disclosed subject matter may be used in the context of VVC.

It is noted that the electronic devices (120) and (130) can include other components (not shown). For example, the electronic device (120) can include a video decoder (not shown) and the electronic device (130) can include a video encoder (not shown) as well.

FIG. 2 shows an exemplary block diagram of a video decoder (210). The video decoder (210) can be included in an electronic device (230). The electronic device (230) can include a receiver (231). The receiver (231) may include receiving circuitry, such as network interface circuitry. The video decoder (210) can be used in the place of the video decoder (110) in the FIG. 1 example.

The receiver (231) may receive one or more coded video sequences to be decoded by the video decoder (210). In an embodiment, one coded video sequence is received at a time, where the decoding of each coded video sequence is independent from the decoding of other coded video sequences. The coded video sequence may be received from a channel (201), which may be a hardware/software link to a storage device which stores the encoded video data. The receiver (231) may receive the encoded video data with other data, for example, coded audio data and/or ancillary data streams, that may be forwarded to their respective using

entities (not depicted). The receiver (231) may separate the coded video sequence from the other data. To combat network jitter, a buffer memory (215) may be coupled in between the receiver (231) and an entropy decoder/parser (220) ("parser (220)" henceforth). In certain applications, the buffer memory (215) is part of the video decoder (210). In others, it can be outside of the video decoder (210) (not depicted). In still others, there can be a buffer memory (not depicted) outside of the video decoder (210), for example to combat network jitter, and in addition another buffer memory (215) inside the video decoder (210), for example to handle playout timing. When the receiver (231) is receiving data from a store/forward device of sufficient bandwidth and controllability, or from an isosynchronous network, the buffer memory (215) may not be needed, or can be small. For use on best effort packet networks such as the Internet, the buffer memory (215) may be required, can be comparatively large and can be advantageously of adaptive size, and may at least partially be implemented in an operating system or similar elements (not depicted) outside of the video decoder (210).

The video decoder (210) may include the parser (220) to reconstruct symbols (221) from the coded video sequence. Categories of those symbols include information used to manage operation of the video decoder (210), and potentially information to control a rendering device such as a render device (212) (e.g., a display screen) that is not an integral part of the electronic device (230) but can be coupled to the electronic device (230), as shown in FIG. 2. The control information for the rendering device(s) may be in the form of Supplemental Enhancement Information (SEI) messages or Video Usability Information (VUI) parameter set fragments (not depicted). The parser (220) may parse/entropy-decode the coded video sequence that is received. The coding of the coded video sequence can be in accordance with a video coding technology or standard, and can follow various principles, including variable length coding, Huffman coding, arithmetic coding with or without context sensitivity, and so forth. The parser (220) may extract from the coded video sequence, a set of subgroup parameters for at least one of the subgroups of pixels in the video decoder, based upon at least one parameter corresponding to the group. Subgroups can include Groups of Pictures (GOPs), pictures, tiles, slices, macroblocks, Coding Units (CUs), blocks, Transform Units (TUs), Prediction Units (PUs) and so forth. The parser (220) may also extract from the coded video sequence information such as transform coefficients, quantizer parameter values, motion vectors, and so forth.

The parser (220) may perform an entropy decoding/parsing operation on the video sequence received from the buffer memory (215), so as to create symbols (221).

Reconstruction of the symbols (221) can involve multiple different units depending on the type of the coded video picture or parts thereof (such as: inter and intra picture, inter and intra block), and other factors. Which units are involved, and how, can be controlled by subgroup control information parsed from the coded video sequence by the parser (220). The flow of such subgroup control information between the parser (220) and the multiple units below is not depicted for clarity.

Beyond the functional blocks already mentioned, the video decoder (210) can be conceptually subdivided into a number of functional units as described below. In a practical implementation operating under commercial constraints, many of these units interact closely with each other and can, at least partly, be integrated into each other. However, for the

purpose of describing the disclosed subject matter, the conceptual subdivision into the functional units below is appropriate.

A first unit is the scaler/inverse transform unit (251). The scaler/inverse transform unit (251) receives a quantized transform coefficient as well as control information, including which transform to use, block size, quantization factor, quantization scaling matrices, etc. as symbol(s) (221) from the parser (220). The scaler/inverse transform unit (251) can output blocks comprising sample values, that can be input into aggregator (255).

In some cases, the output samples of the scaler/inverse transform unit (251) can pertain to an intra coded block. The intra coded block is a block that is not using predictive information from previously reconstructed pictures, but can use predictive information from previously reconstructed parts of the current picture. Such predictive information can be provided by an intra picture prediction unit (252). In some cases, the intra picture prediction unit (252) generates a block of the same size and shape of the block under reconstruction, using surrounding already reconstructed information fetched from the current picture buffer (258). The current picture buffer (258) buffers, for example, partly reconstructed current picture and/or fully reconstructed current picture. The aggregator (255), in some cases, adds, on a per sample basis, the prediction information the intra prediction unit (252) has generated to the output sample information as provided by the scaler/inverse transform unit (251).

In other cases, the output samples of the scaler/inverse transform unit (251) can pertain to an inter coded, and potentially motion compensated, block. In such a case, a motion compensation prediction unit (253) can access reference picture memory (257) to fetch samples used for prediction. After motion compensating the fetched samples in accordance with the symbols (221) pertaining to the block, these samples can be added by the aggregator (255) to the output of the scaler/inverse transform unit (251) (in this case called the residual samples or residual signal) so as to generate output sample information. The addresses within the reference picture memory (257) from where the motion compensation prediction unit (253) fetches prediction samples can be controlled by motion vectors, available to the motion compensation prediction unit (253) in the form of symbols (221) that can have, for example X, Y, and reference picture components. Motion compensation also can include interpolation of sample values as fetched from the reference picture memory (257) when sub-sample exact motion vectors are in use, motion vector prediction mechanisms, and so forth.

The output samples of the aggregator (255) can be subject to various loop filtering techniques in the loop filter unit (256). Video compression technologies can include in-loop filter technologies that are controlled by parameters included in the coded video sequence (also referred to as coded video bitstream) and made available to the loop filter unit (256) as symbols (221) from the parser (220). Video compression can also be responsive to meta-information obtained during the decoding of previous (in decoding order) parts of the coded picture or coded video sequence, as well as responsive to previously reconstructed and loop-filtered sample values.

The output of the loop filter unit (256) can be a sample stream that can be output to the render device (212) as well as stored in the reference picture memory (257) for use in future inter-picture prediction.

Certain coded pictures, once fully reconstructed, can be used as reference pictures for future prediction. For

example, once a coded picture corresponding to a current picture is fully reconstructed and the coded picture has been identified as a reference picture (by, for example, the parser (220)), the current picture buffer (258) can become a part of the reference picture memory (257), and a fresh current picture buffer can be reallocated before commencing the reconstruction of the following coded picture.

The video decoder (210) may perform decoding operations according to a predetermined video compression technology or a standard, such as ITU-T Rec. H.265. The coded video sequence may conform to a syntax specified by the video compression technology or standard being used, in the sense that the coded video sequence adheres to both the syntax of the video compression technology or standard and the profiles as documented in the video compression technology or standard. Specifically, a profile can select certain tools as the only tools available for use under that profile from all the tools available in the video compression technology or standard. Also necessary for compliance can be that the complexity of the coded video sequence is within bounds as defined by the level of the video compression technology or standard. In some cases, levels restrict the maximum picture size, maximum frame rate, maximum reconstruction sample rate (measured in, for example megasamples per second), maximum reference picture size, and so on. Limits set by levels can, in some cases, be further restricted through Hypothetical Reference Decoder (HRD) specifications and metadata for HRD buffer management signaled in the coded video sequence.

In an embodiment, the receiver (231) may receive additional (redundant) data with the encoded video. The additional data may be included as part of the coded video sequence(s). The additional data may be used by the video decoder (210) to properly decode the data and/or to more accurately reconstruct the original video data. Additional data can be in the form of, for example, temporal, spatial, or signal noise ratio (SNR) enhancement layers, redundant slices, redundant pictures, forward error correction codes, and so on.

FIG. 3 shows an exemplary block diagram of a video encoder (303). The video encoder (303) is included in an electronic device (320). The electronic device (320) includes a transmitter (340) (e.g., transmitting circuitry). The video encoder (303) can be used in the place of the video encoder (103) in the FIG. 1 example.

The video encoder (303) may receive video samples from a video source (301) (that is not part of the electronic device (320) in the FIG. 3 example) that may capture video image(s) to be coded by the video encoder (303). In another example, the video source (301) is a part of the electronic device (320).

The video source (301) may provide the source video sequence to be coded by the video encoder (303) in the form of a digital video sample stream that can be of any suitable bit depth (for example: 8 bit, 10 bit, 12 bit, . . .), any colorspace (for example, BT.601 Y CrCb, RGB, . . .), and any suitable sampling structure (for example Y CrCb 4:2:0, Y CrCb 4:4:4). In a media serving system, the video source (301) may be a storage device storing previously prepared video. In a videoconferencing system, the video source (301) may be a camera that captures local image information as a video sequence. Video data may be provided as a plurality of individual pictures that impart motion when viewed in sequence. The pictures themselves may be organized as a spatial array of pixels, wherein each pixel can

comprise one or more samples depending on the sampling structure, color space, etc., in use. The description below focuses on samples.

According to an embodiment, the video encoder (303) may code and compress the pictures of the source video sequence into a coded video sequence (343) in real time or under any other time constraints as required. Enforcing appropriate coding speed is one function of a controller (350). In some embodiments, the controller (350) controls other functional units as described below and is functionally coupled to the other functional units. The coupling is not depicted for clarity. Parameters set by the controller (350) can include rate control related parameters (picture skip, quantizer, lambda value of rate-distortion optimization techniques, . . .), picture size, group of pictures (GOP) layout, maximum motion vector search range, and so forth. The controller (350) can be configured to have other suitable functions that pertain to the video encoder (303) optimized for a certain system design.

In some embodiments, the video encoder (303) is configured to operate in a coding loop. As an oversimplified description, in an example, the coding loop can include a source coder (330) (e.g., responsible for creating symbols, such as a symbol stream, based on an input picture to be coded, and a reference picture(s)), and a (local) decoder (333) embedded in the video encoder (303). The decoder (333) reconstructs the symbols to create the sample data in a similar manner as a (remote) decoder also would create. The reconstructed sample stream (sample data) is input to the reference picture memory (334). As the decoding of a symbol stream leads to bit-exact results independent of decoder location (local or remote), the content in the reference picture memory (334) is also bit exact between the local encoder and remote encoder. In other words, the prediction part of an encoder “sees” as reference picture samples exactly the same sample values as a decoder would “see” when using prediction during decoding. This fundamental principle of reference picture synchronicity (and resulting drift, if synchronicity cannot be maintained, for example because of channel errors) is used in some related arts as well.

The operation of the “local” decoder (333) can be the same as of a “remote” decoder, such as the video decoder (210), which has already been described in detail above in conjunction with FIG. 2. Briefly referring also to FIG. 2, however, as symbols are available and encoding/decoding of symbols to a coded video sequence by an entropy coder (345) and the parser (220) can be lossless, the entropy decoding parts of the video decoder (210), including the buffer memory (215), and parser (220) may not be fully implemented in the local decoder (333).

In an embodiment, a decoder technology except the parsing/entropy decoding that is present in a decoder is present, in an identical or a substantially identical functional form, in a corresponding encoder. Accordingly, the disclosed subject matter focuses on decoder operation. The description of encoder technologies can be abbreviated as they are the inverse of the comprehensively described decoder technologies. In certain areas a more detail description is provided below.

During operation, in some examples, the source coder (330) may perform motion compensated predictive coding, which codes an input picture predictively with reference to one or more previously coded picture from the video sequence that were designated as “reference pictures.” In this manner, the coding engine (332) codes differences between pixel blocks of an input picture and pixel blocks of

reference picture(s) that may be selected as prediction reference(s) to the input picture.

The local video decoder (333) may decode coded video data of pictures that may be designated as reference pictures, based on symbols created by the source coder (330). Operations of the coding engine (332) may advantageously be lossy processes. When the coded video data may be decoded at a video decoder (not shown in FIG. 3), the reconstructed video sequence typically may be a replica of the source video sequence with some errors. The local video decoder (333) replicates decoding processes that may be performed by the video decoder on reference pictures and may cause reconstructed reference pictures to be stored in the reference picture memory (334). In this manner, the video encoder (303) may store copies of reconstructed reference pictures locally that have common content as the reconstructed reference pictures that will be obtained by a far-end video decoder (absent transmission errors).

The predictor (335) may perform prediction searches for the coding engine (332). That is, for a new picture to be coded, the predictor (335) may search the reference picture memory (334) for sample data (as candidate reference pixel blocks) or certain metadata such as reference picture motion vectors, block shapes, and so on, that may serve as an appropriate prediction reference for the new pictures. The predictor (335) may operate on a sample block-by-pixel block basis to find appropriate prediction references. In some cases, as determined by search results obtained by the predictor (335), an input picture may have prediction references drawn from multiple reference pictures stored in the reference picture memory (334).

The controller (350) may manage coding operations of the source coder (330), including, for example, setting of parameters and subgroup parameters used for encoding the video data.

Output of all aforementioned functional units may be subjected to entropy coding in the entropy coder (345). The entropy coder (345) translates the symbols as generated by the various functional units into a coded video sequence, by applying lossless compression to the symbols according to technologies such as Huffman coding, variable length coding, arithmetic coding, and so forth.

The transmitter (340) may buffer the coded video sequence(s) as created by the entropy coder (345) to prepare for transmission via a communication channel (360), which may be a hardware/software link to a storage device which would store the encoded video data. The transmitter (340) may merge coded video data from the video encoder (303) with other data to be transmitted, for example, coded audio data and/or ancillary data streams (sources not shown).

The controller (350) may manage operation of the video encoder (303). During coding, the controller (350) may assign to each coded picture a certain coded picture type, which may affect the coding techniques that may be applied to the respective picture. For example, pictures often may be assigned as one of the following picture types:

An Intra Picture (I picture) may be one that may be coded and decoded without using any other picture in the sequence as a source of prediction. Some video codecs allow for different types of intra pictures, including, for example Independent Decoder Refresh ("IDR") Pictures. A person skilled in the art is aware of those variants of I pictures and their respective applications and features.

A predictive picture (P picture) may be one that may be coded and decoded using intra prediction or inter prediction using at most one motion vector and reference index to predict the sample values of each block.

A bi-directionally predictive picture (B Picture) may be one that may be coded and decoded using intra prediction or inter prediction using at most two motion vectors and reference indices to predict the sample values of each block. Similarly, multiple-predictive pictures can use more than two reference pictures and associated metadata for the reconstruction of a single block.

Source pictures commonly may be subdivided spatially into a plurality of sample blocks (for example, blocks of 4×4, 8×8, 4×8, or 16×16 samples each) and coded on a block-by-block basis. Blocks may be coded predictively with reference to other (already coded) blocks as determined by the coding assignment applied to the blocks' respective pictures. For example, blocks of I pictures may be coded non-predictively or they may be coded predictively with reference to already coded blocks of the same picture (spatial prediction or intra prediction). Pixel blocks of P pictures may be coded predictively, via spatial prediction or via temporal prediction with reference to one previously coded reference picture. Blocks of B pictures may be coded predictively, via spatial prediction or via temporal prediction with reference to one or two previously coded reference pictures.

The video encoder (303) may perform coding operations according to a predetermined video coding technology or standard, such as ITU-T Rec. H.265. In its operation, the video encoder (303) may perform various compression operations, including predictive coding operations that exploit temporal and spatial redundancies in the input video sequence. The coded video data, therefore, may conform to a syntax specified by the video coding technology or standard being used.

In an embodiment, the transmitter (340) may transmit additional data with the encoded video. The source coder (330) may include such data as part of the coded video sequence. Additional data may comprise temporal/spatial/SNR enhancement layers, other forms of redundant data such as redundant pictures and slices, SEI messages, VUI parameter set fragments, and so on.

A video may be captured as a plurality of source pictures (video pictures) in a temporal sequence. Intra-picture prediction (often abbreviated to intra prediction) makes use of spatial correlation in a given picture, and inter-picture prediction makes use of the (temporal or other) correlation between the pictures. In an example, a specific picture under encoding/decoding, which is referred to as a current picture, is partitioned into blocks. When a block in the current picture is similar to a reference block in a previously coded and still buffered reference picture in the video, the block in the current picture can be coded by a vector that is referred to as a motion vector. The motion vector points to the reference block in the reference picture, and can have a third dimension identifying the reference picture, in case multiple reference pictures are in use.

In some embodiments, a bi-prediction technique can be used in the inter-picture prediction. According to the bi-prediction technique, two reference pictures, such as a first reference picture and a second reference picture that are both prior in decoding order to the current picture in the video (but may be in the past and future, respectively, in display order) are used. A block in the current picture can be coded by a first motion vector that points to a first reference block in the first reference picture, and a second motion vector that points to a second reference block in the second reference picture. The block can be predicted by a combination of the first reference block and the second reference block.

Further, a merge mode technique can be used in the inter-picture prediction to improve coding efficiency.

According to some embodiments of the disclosure, predictions, such as inter-picture predictions and intra-picture predictions, are performed in the unit of blocks, such as a polygon-shaped or triangular block. For example, according to the HEVC standard, a picture in a sequence of video pictures is partitioned into coding tree units (CTU) for compression, the CTUs in a picture have the same size, such as 64×64 pixels, 32×32 pixels, or 16×16 pixels. In general, a CTU includes three coding tree blocks (CTBs), which are one luma CTB and two chroma CTBs. Each CTU can be recursively quadtree split into one or multiple coding units (CUs). For example, a CTU of 64×64 pixels can be split into one CU of 64×64 pixels, or 4 CUs of 32×32 pixels, or 16 CUs of 16×16 pixels. In an example, each CU is analyzed to determine a prediction type for the CU, such as an inter prediction type or an intra prediction type. The CU is split into one or more prediction units (PUs) depending on the temporal and/or spatial predictability. Generally, each PU includes a luma prediction block (PB), and two chroma PBs. In an embodiment, a prediction operation in coding (encoding/decoding) is performed in the unit of a prediction block. Using a luma prediction block as an example of a prediction block, the prediction block includes a matrix of values (e.g., luma values) for pixels, such as 8×8 pixels, 16×16 pixels, 8×16 pixels, 16×8 pixels, and the like.

It is noted that the video encoders (103) and (303), and the video decoders (110) and (210) can be implemented using any suitable technique. In an embodiment, the video encoders (103) and (303) and the video decoders (110) and (210) can be implemented using one or more integrated circuits. In another embodiment, the video encoders (103) and (303), and the video decoders (110) and (210) can be implemented using one or more processors that execute software instructions.

The disclosure includes embodiments related to methods and systems of a mesh vertex position compression using a temporal prediction.

A mesh can include several polygons that describe a surface of a volumetric object. Each polygon of the mesh can be defined by vertices of the corresponding polygon in a three-dimensional (3D) space and information of how the vertices are connected, which can be referred to as connectivity information. In some embodiments, vertex attributes, such as colors, normals, etc., can be associated with the mesh vertices. Attributes (or vertex attributes) can also be associated with the surface of the mesh by exploiting mapping information that parameterizes the mesh with two-dimensional (2D) attribute maps. Such mapping can usually be described by a set of parametric coordinates, referred to as UV coordinates or texture coordinates, associated with the mesh vertices. 2D attribute maps can be used to store high resolution attribute information such as texture, normals, displacements etc. Such information can be used for various purposes such as texture mapping and shading.

A dynamic mesh sequence may require a large amount of data since the dynamic mesh can include a significant amount of information changing over time. Therefore, efficient compression technologies are required to store and transmit such contents. Mesh compression standards, such as IC, MESHGRID, and FAMC, were previously developed by MPEG to address dynamic meshes with a constant connectivity, a time varying geometry, and vertex attributes. However, these standards may not consider time varying attribute maps and connectivity information. DCC (Digital Content Creation) tools usually generate such dynamic

meshes. However, it can be challenging for volumetric acquisition techniques to generate a constant connectivity dynamic mesh, especially under real time constraints. This type of content (e.g., a constant connectivity dynamic mesh) may not be supported by existing standards. MPEG is planning to develop a new mesh compression standard to directly handle dynamic meshes with time varying connectivity information and optionally time varying attribute maps. The new mesh compression standard targets lossy and lossless compression for various applications, such as real-time communications, a storage, a free viewpoint video, Augmented Reality (AR), and Virtual Reality (VR). Functionalities, such as a random access and a scalable/progressive coding, can also be considered.

Mesh geometry information can include vertex connectivity information, 3D coordinates, and 2D texture coordinates, etc. Compression of vertex 3D coordinates, which may also be referred to as vertex positions, can be important because, in many cases, compression of the vertex 3D coordinates can consume a significant portion of the entire geometry related data.

A dynamic mesh sequence M at a time instance t can be denoted as $M(t)$. $M(t)$ can be referred to as a positionally tracked frame, if there is a mapping (or mapping operation) f from vertex positions of $M(t)$ to vertex positions at another time instance $M(t_0)$, where t and t_0 are different time instances. Accordingly, $M(t_0)$ can be referred to as a reference frame of the frame $M(t)$, and corresponding vertices in the reference frame can be referred to as reference vertices of the vertices in $M(t)$. $M(t)$ can correspond to a mesh at a first time instance, such as a current instance, and $M(t_0)$ can correspond to the mesh at a second time instance, such as a previous time instance.

In this disclosure, methods and/or systems are proposed for vertex position compression. Note that the methods and/or systems can be applied individually or in various combinations. Further, the disclosed methods and systems are not limited to the vertex position compression. The disclosed methods and systems can also be applied to, for example, two-dimensional (2D) texture coordinate compression or a more general temporal prediction-based scheme.

For a vertex V in a positionally tracked frame $M(t)$, neighbors of the vertex can also be vertices that are connected to V through edges, and these vertices are referred to as neighbor vertices (or neighboring vertices) of V . For example, as shown in FIG. 4, a vertex A can have four neighbor vertices, which are C , D , E , and B . A vertex E can have five neighbor vertices, which are A , B , F , H , and D .

FIG. 5 is a schematic diagram of an exemplary vertex position compression apparatus (500) according to some embodiments of the disclosure. As shown in FIG. 5, the vertex position compression apparatus (500) can include an vertex ordering module (502) configured to order vertices of a mesh, a position prediction module (504) configured to calculate prediction positions of vertices, a prediction index coding module (506) configured to code (or determine) a position prediction index (or a position prediction mode, if necessary), and a prediction residue coding module (508) configured to code a position prediction residue. It should be noted that the mesh processed in the vertex position compression apparatus (500) can be a 3D mesh or a 2D mesh. For example, the mesh can be represented in a 3D coordinate system (e.g., x , y , and z) or in a 2D coordinate system (e.g., U and V).

The term module in this disclosure may refer to a software module, a hardware module, or a combination thereof. A software module (e.g., computer program) may be devel-

13

oped using a computer programming language. A hardware module may be implemented using processing circuitry and/or memory. Each module can be implemented using one or more processors (or processors and memory). Likewise, a processor (or processors and memory) can be used to implement one or more modules. Moreover, each module can be part of an overall module that includes the functionalities of the module.

In the disclosure, vertices of a positionally tracked frame (or frame) $M(t)$ can be ordered. An order of the vertices in the $M(t)$ can be traversed according to an Edgebreaker algorithm or other partition algorithms.

FIGS. 6A and 6B show an exemplary ordering of triangles and vertices in a frame $M(t)$ based on the Edgebreaker algorithm. FIG. 6A shows five exemplary patch configurations of the Edgebreaker algorithm. As shown in FIG. 6A, V_i is a patch center vertex and T is a current triangle. An active gate (or current triangle) in each patch can be denoted as T . In patch C, a complete triangle that fans (or rotates) around V can be provided. In patch L, one or more missing triangles can be positioned to the left of the active gate T . In patch R, one or more missing triangles can be positioned to the right of the active gate T . In patch E, V_i is only adjacent to T . In patch S, one or more missing triangles can be positioned at positions other than to the left or the right of the active gate T . FIG. 6B shows an exemplary traversal of a frame (600) in which triangles of the frame (600) can be ordered based on the traversal of the Edgebreaker algorithm. As shown in FIG. 6B, the triangles in the frame (600) can be traversed along a spiraling triangle-spanning-tree. For example, the traversal can start with a triangle (602) of the type C (or patch C). The traversal can then proceed along a branch adjacent to a right edge of the triangle (e.g., (602)). The traversal can stop when a triangle of the type E (e.g., (604)) is reached. According to the Edgebreaker algorithm, the triangles of the frame (600) can be traversed (or ordered) in a sequence of CRSRLECRRLLE, which can be shown in FIG. 6B. Vertices in each triangle of the frame (600) can also be ordered based on the order of the triangles.

In the disclosure, a temporal prediction can be applied to predict a vertex in a current frame based on a reference vertex in a reference frame. The current frame can correspond to a mesh at a first time instance, such as a current time instance. The reference frame can correspond to the mesh at a second time instance, such as a previous time instance. In some embodiments, connectivity of vertices of the mesh in the current frame can be different from connectivity of the vertices of the mesh in the reference frame. For a vertex V in a positionally tracked frame (or a current frame) $M(t)$, a position of the vertex V can be estimated by a position of a reference vertex $f(V)$ in a reference frame (e.g., $M(t_0)$), to is a different time instance from the time instance t , where f is a mapping operation between $M(t)$ and the reference frame. In some embodiments, the vertex V and the reference vertex $f(V)$ in the reference frame are collocated. Thus, the reference vertex can have a same relative position in the reference frame as the vertex in the current frame $M(t)$. When the vertex V is predicted by the reference vertex $f(V)$, an estimation error E can be determined as a difference between the positions of V and the position of $f(V)$ in equation (1).

$$E = V - f(V) \quad \text{Eq. (1)}$$

As each vertex in the frame $M(t)$ can have 3D coordinates, 3D coordinate components of the estimation error E can be provided based on equation (1). For example, assume subscripts x , y , z denote 3D coordinates in an xyz space, then

14

the 3D coordinate components of the estimation error E can be provided in equations (2)-(4).

$$E_x = V_x - (f(V))_x \quad \text{Eq. (2)}$$

$$E_y = V_y - (f(V))_y \quad \text{Eq. (3)}$$

$$E_z = V_z - (f(V))_z \quad \text{Eq. (4)}$$

The estimation error E of the vertex V can be predicted, or otherwise determined, from neighbors (or neighboring vertices) of the vertex V . For a neighbor vertex (or neighboring vertex) of V , if the neighboring vertex has been coded and can be used for prediction, an estimation error of the neighboring vertex can be applied to predict E .

Assume V has N neighbor vertices (or neighboring vertices) $V_1, V_2, \dots, V_N$ that have been coded and can be used for prediction. In an example, an order of the neighboring vertices $V_1, V_2, \dots, V_N$ can be determined based on the Edgebreaker algorithm or other partition algorithms. For a neighbor vertex V_i , an estimation error of the neighbor vertex V_i can be determined as $E_i = V_i - f(V_i)$, for $i=1, 2, \dots, N$. $f(V_i)$ can be a reference vertex for the neighbor vertex V_i in a reference frame. E_i can also be referred to as neighboring estimation errors associated with the vertex V and each E_i can be a prediction candidate of E . If duplicates are determined in predicted values (or prediction candidates) E , such duplicates can be removed from the list $E_1, E_2, \dots, E_N$.

If $N \geq 2$, more than one prediction candidate (or predicted value) E_i are available. In an embodiment, an average of the predicted values E_i can be used as an additional predictor and set as E_0 . For example, E_0 can be defined in equation (5) as follows:

$$E_0 = (E_1 + E_2 + \dots + E_N) / N \quad \text{Eq. (5)}$$

If the average E_0 is equal to one of the predicted values $E_1, E_2, \dots, E_N$, either the average or the duplicated predicted value can be removed.

On an encoder side, each E_i can be compared with E , where $0 \leq i \leq N$. An estimation error can be selected from $E_0, E_1, E_2, \dots, E_N$. The selected estimation error can have a minimum error (or minimum difference) between the E and each of the prediction values (or predicted values or prediction candidates) $E_0, E_1, E_2, \dots, E_N$. In some embodiments, the minimum error can be measured by L^0 norm, L^1 norm, L^2 norm, or some other norm. For example, the L^0 norm can be determined in equation (6) as follows:

$$L^0 \text{ norm} = \sqrt{(E_{0x} - E_x)^2 + (E_{0y} - E_y)^2 + (E_{0z} - E_z)^2} \quad \text{Eq. (6)}$$

where E_x, E_y , and E_z are coordinates of E in a xyz space, and E_{0x}, E_{0y} , and E_{0z} are coordinates of E_0 in the xyz space. In an example, the selected estimation error can be one of the E_i that corresponds to a minimum error. In an example, the selected estimation error can be E itself. For convenience, an index -1 can be used to denote E , such as $E_{-1} = E$. Thus, the selected estimation error can be stored (or identified) as an index whose value is between -1 and N . On a decoder side, the index of the selection that indicated the selected estimation error can be decoded and the selected predictor (or selected estimation error) can be recovered from a list of predictors $E_{-1}, E_0, E_1, E_2, \dots, E_N$.

In an embodiment, an upper limit, M , can be set. The upper limit M can indicate how many prediction candidates can be considered (or applied) in the prediction list. In an example, if $N > M$, then only the first M prediction candidates $E_1, E_2, \dots, E_M$ are considered to predict E . Accordingly, E_0 can be defined in equation (7) as follows:

$$E_0 = (E_1 + E_2 + \dots + E_M) / M \quad \text{Eq. (7)}$$

15

M can be an integer, such as 4. Thus, up to M (e.g., 4) prediction candidates can be considered. Note that the average prediction E_0 can be put in different positions in the candidate list. For example, the average prediction E_0' can be a first predictor in the prediction candidate list. In an example, the average prediction E_0' can be a last predictor in the prediction candidate list. In an example, the average prediction E_0' can be positioned in the prediction values $E_1', E_2, \dots, E_M$.

An intra frame prediction can be applied to predict the vertex position V. The intra frame prediction can be a delta prediction, a parallelogram prediction, or the like. In the delta prediction, a predictor can be determined as a neighboring vertex of the vertex V. In the parallelogram prediction, a prediction vertex can be determined based on a prediction triangle of a plurality of triangles of the mesh. The prediction triangle shares an edge with a triangle of the plurality of triangles that includes the vertex V. The prediction vertex can be opposite to the shared edge. The prediction vertex and the prediction triangle can form a parallelogram.

A prediction error from the intra frame prediction can be compared with the selected estimation error E_i , where $-1 \leq i \leq N$. The prediction error of the intra frame prediction can indicate a true position of the vertex V and a predicted position value based on the intra frame prediction. When the intra frame prediction provides a smaller error (e.g., the prediction error is smaller than the selected estimation error), then the intra frame prediction is used, and an intra prediction mode can be signaled. Otherwise, when the intra frame prediction provides a larger error than the selected estimation error E_i , the temporal prediction can be used, and an inter prediction mode (e.g., temporal prediction) can be signaled.

A mixed prediction can be applied to predict a position of a vertex V. For example, an average of the intra frame prediction and the temporal prediction can be applied as a prediction of the vertex position V.

In the disclosure, the prediction index needs to be coded only if more than one prediction candidates exist. For example, when two or more prediction candidates are available for a current vertex, an index indicating a selected prediction candidate can be coded at an encoder side. At a decoder side, the decoder can determine the two or more prediction candidates with a same order as the encoders. The decoder can decode the coded prediction index and reconstruct the selected prediction candidate based on the prediction index from the two or more prediction candidates.

In an embodiment, if no prediction candidate is determined, or only one prediction candidate is determined, the prediction index may not be required to be coded. Therefore, the prediction value can be the current vertex itself or the only prediction candidate. In an embodiment, the prediction index can always be coded regardless how many predictor candidates are available. When no predictor is available, the value of the signaled index may not affect a decoding process in the decoder side.

When more than one prediction candidate are determined for a current vertex, such as $N \geq 1$, the prediction index can be coded using a fixed length coding. For example, if 3 prediction candidates are determined for the current vertex, then 4 possible prediction indices, 0, 1, 2, and 3 (0 indicates an average value of candidates 1, 2, and 3) can be required. Thus, a two-digit binary number can be applied to represent each of the four prediction indices. It should be noted that

16

different vertices of a triangle mesh could use different fixed lengths. For example, if another vertex has 7 prediction candidates, then the other vertex can use a three-digit binary number for a prediction index representation. An output from the fixed length coding can be further compressed by an entropy coding, such as arithmetic coding.

Alternatively, the prediction index can be coded using a variable length coding. For example, if 4 prediction candidates are determined for a current vertex, then 5 possible prediction indices, 0, 1, 2, 3, and 4 can be required. A variable length code of 0, 100, 101, 110, and 111 can be assigned to represent the 5 prediction indices 0, 1, 2, 3, and 4, respectively. Alternatively, a variable length code of 1, 01, 001, 0001, and 00001 can be applied to represent 5 prediction indices of 0, 1, 2, 3, and 4, respectively. Note that different vertices of a triangle mesh could use different variable lengths. An output from the variable length coding can be further compressed by an entropy coding, such as arithmetic coding.

When an upper limit M is set to limit a maximum number of prediction candidates, and if more than M prediction candidates are available for a vertex V, then only the first M prediction candidates may be used to predict the vertex V. Thus, (M+1) possible prediction indices can be coded, where M indicates the first M prediction candidates and 1 indicates an average of the first M prediction candidates. If duplicates are determined in predicted values, the duplicates can be removed. Accordingly, a possibility of prediction indices can also be reduced. Prediction index can be coded by a fixed length coding, a variable length coding, a differential coding, or the like.

When the intra frame prediction is also a prediction candidate, a one-bit binary flag can be used to signal a position prediction mode. The position prediction mode can indicate whether an inter frame prediction (e.g., the temporal prediction) or an intra frame prediction is applied.

When the average of the intra frame prediction and the inter frame prediction is also a prediction candidate, a 3-symbol flag can be used to signal the position prediction mode. The position prediction mode can indicate whether an inter frame prediction, an intra frame prediction, or the average of the intra frame prediction and the inter frame prediction is applied.

In the disclosure, a prediction residue of the vertex position can be coded (or determined). When the temporal prediction is applied, if the prediction index j is -1 (-1 means E itself), the prediction residue can be determined as E (e.g., $V - f(V)$). Thus, the vertex V can be reconstructed as $V = f(V) + E$. Otherwise (e.g., index is 0, or one of 1, 2, ... N), the prediction residue can be determined as $E - E_j$, where j is the prediction index. Accordingly, E can be recovered as $E = E_j + \text{the prediction residue}$ (e.g., $E - E_j$), and the vertex V can further be reconstructed as $V = f(V) + (E_j + (E - E_j))$.

When the intra frame prediction is applied, the prediction residue can be determined as a difference between a true position value of the vertex V and a predicted position value.

When the average of the intra frame prediction and the inter frame prediction is applied, the prediction residue can be determined as a difference between a true position value of a vertex V and a predicted position value.

In some embodiments, the prediction residue can be coded using a coding algorithm, such as a fixed length coding, an exponential-Golomb coding, an arithmetic coding, or the like. In some embodiments, the prediction residue can go through a compactization transform, such as a fast Fourier transform (FFT), a discrete cosine transform (DCT), a discrete sine transform (DST), a discrete wavelet transform

(DWT), or the like. An output from the compactization transform can be coded using a coding algorithm, such as a fixed length coding, an exponential-Golomb coding, an arithmetic coding, or the like.

FIG. 7 shows a flow chart outlining a process (700) according to an embodiment of the disclosure. The process (700) can be used in an encoder, such as a video encoder. In various embodiments, the process (700) is executed by processing circuitry, such as the processing circuitry that performs functions of the video encoder (103), the processing circuitry that performs functions of the video encoder (303), and the like. In some embodiments, the process (700) is implemented in software instructions, thus when the processing circuitry executes the software instructions, the processing circuitry performs the process (700). The process starts at (S701) and proceeds to (S710).

At (S710), a plurality of neighboring vertices of a current vertex is determined in a current frame of a mesh in one of a two-dimensional model and a three-dimensional model. The current vertex and the plurality of the neighboring vertices are included in the current frame and corresponds to the mesh at a first time instance. Each of the plurality of neighboring vertices is connected to the current vertex through a respective edge in the mesh.

At (S720), a plurality of neighboring estimation errors of the plurality of neighboring vertices of the current vertex is determined. Each of the plurality of neighboring estimation errors indicates a difference between a reference vertex of a corresponding one of the plurality of neighboring vertices in a reference frame of the mesh and the corresponding one of the plurality of neighboring vertices in the current frame. The reference frame corresponds to the mesh at a second time instance.

At (S730), a prediction residue of the current vertex is determined based on the plurality of neighboring estimation errors of the plurality of neighboring vertices.

At (S740), prediction information of the current vertex is generated based on the determined prediction residue of the current vertex.

In some embodiments, the reference vertex of the corresponding one of the plurality of neighboring vertices is positioned at a same relative position in the reference frame as the current vertex in the current frame, where the reference frame and the current frame are generated at different time instances.

In an example, an average neighboring estimation error of the plurality of neighboring estimation errors is determined. The prediction residue of the current vertex is determined based on the average neighboring estimation error.

In an example, to determine the prediction residue of the current vertex, an estimation error of the current vertex is determined based on a difference between a reference vertex of the current vertex and the current vertex. A prediction list of the current vertex is determined. Predictors of the prediction list include the estimation error of the current vertex, the average neighboring estimation error subsequent to the estimation error of the current vertex, and the plurality of neighboring estimation errors subsequent to the average neighboring estimation error. A respective prediction index is associated with each of the predictors in the prediction list.

In some embodiments, the plurality of neighboring vertices is ordered based on an Edgebreaker algorithm in which the plurality of neighboring vertices is traversed in a spiraling triangle-spanning-tree order.

In an example, to determine the prediction residue of the current vertex, a difference between the average neighboring estimation error and the estimation error of the current

vertex is determined. A difference between each of the plurality of neighboring estimation errors and the estimation error of the current vertex is also determined. A neighboring estimation error from the average neighboring estimation error and the plurality of neighboring estimation errors is selected. The selected neighboring estimation error has a minimum difference. The prediction residue of the current vertex is determined as one of (i) the estimation error of the current vertex and (ii) the selected neighboring estimation error.

In an example, to determine the prediction residue of the current vertex, a prediction error is determined based on one of the plurality of neighboring vertices of the current vertex according to an inter frame prediction. The prediction error is compared with each of the predictors in the prediction list. The prediction residue is determined as the prediction error in response to the prediction error being smaller than the predictors in the prediction list.

In an example, to determine the prediction residue of the current vertex, the prediction residue is determined as a mixed estimation error. The mixed estimation error includes one of (i) an average of the selected neighboring estimation error and the prediction error and (ii) an average of the estimation error of the current vertex and the prediction error.

In an embodiment, the inter frame prediction further includes one of a delta prediction that is based on a difference between a first prediction vertex and the current vertex, where the first prediction vertex includes the one of the plurality of neighboring vertices of the current vertex. The inter frame prediction can also include a parallelogram prediction in which a second prediction vertex is determined based on a prediction triangle of a plurality of triangles of the mesh. The prediction triangle shares an edge with a triangle of the plurality of triangles that includes the current vertex. The second prediction vertex is opposite to the shared edge, and the second prediction vertex and the prediction triangle form a parallelogram.

In an embodiment, the prediction information of the current vertex further includes a flag. The flag indicates that the prediction residue is one of (i) the estimation error of the current vertex or the selected neighboring estimation error based on a first value of the flag, (ii) the prediction error based on a second value of the flag, and (iii) the mixed estimation error based on a third value of the flag.

In an embodiment, the prediction information of the current vertex further includes index information indicating which one of the predictors in the prediction list is the prediction residue based on the flag being the first value. The prediction information also includes prediction residue information. In an example, based on the flag being the first value, the prediction residue information indicates (i) the estimation error of the current vertex in response to the index information indicating that the prediction residue is the estimation error of the current vertex, and (ii) the selected neighboring estimation error in response to the index information indicating that the prediction residue is the selected neighboring estimation error. Based on the flag being the second value, the prediction residue information indicates a difference between the current vertex and a prediction vertex indicated by the prediction error. Based on the flag being the third value, the prediction residue information indicates a difference between the current vertex and a prediction vertex indicated by the mixed estimation error.

Then, the process proceeds to (S799) and terminates.

The process (700) can be suitably adapted. Step(s) in the process (700) can be modified and/or omitted. Additional step(s) can be added. Any suitable order of implementation can be used.

FIG. 8 shows a flow chart outlining a process (800) according to an embodiment of the disclosure. The process (800) can be used in a decoder, such as a video decoder. In various embodiments, the process (800) is executed by processing circuitry, such as the processing circuitry that performs functions of the video decoder (110), the processing circuitry that performs functions of the video decoder (210), and the like. In some embodiments, the process (800) is implemented in software instructions, thus when the processing circuitry executes the software instructions, the processing circuitry performs the process (800). The process starts at (S801) and proceeds to (S810).

At (S810), coding information of a current frame of a mesh in one of a two-dimensional model and a three-dimensional model is received. The current frame includes a plurality of vertices and corresponds to the mesh at a first time instance. The coding information includes index information of a current vertex in the plurality of vertices. The index information indicates a prediction residue of the current vertex.

At (S820), a plurality of neighboring vertices of the current vertex is determined from the plurality of vertices, where each of the plurality of neighboring vertices is connected to the current vertex through a respective edge.

At (S830), a plurality of neighboring estimation errors of the plurality of neighboring vertices of the current vertex is determined. Each of the plurality of neighboring estimation errors indicates a difference between a reference vertex of a corresponding one of the plurality of neighboring vertices in a reference frame of the mesh and the corresponding one of the plurality of neighboring vertices in the current frame. The reference frame corresponds to the mesh at a second time instance.

At (S840), the prediction residue of the current vertex is determined based on (i) the index information and (ii) the plurality of neighboring estimation errors of the plurality of neighboring vertices.

At (S850), the current vertex is reconstructed based on the determined prediction residue of the current vertex.

Then, the process proceeds to (S899) and terminates.

The process (800) can be suitably adapted. Step(s) in the process (800) can be modified and/or omitted. Additional step(s) can be added. Any suitable order of implementation can be used.

The techniques described above, can be implemented as computer software using computer-readable instructions and physically stored in one or more computer-readable media, such as a non-transitory computer-readable storage medium. For example, FIG. 9 shows a computer system (900) suitable for implementing certain embodiments of the disclosed subject matter.

The computer software can be coded using any suitable machine code or computer language, that may be subject to assembly, compilation, linking, or like mechanisms to create code comprising instructions that can be executed directly, or through interpretation, micro-code execution, and the like, by processing circuitry such as one or more computer central processing units (CPUs), Graphics Processing Units (GPUs), and the like.

The instructions can be executed on various types of computers or components thereof, including, for example, personal computers, tablet computers, servers, smartphones, gaming devices, internet of things devices, and the like.

The components shown in FIG. 9 for computer system (900) are exemplary in nature and are not intended to suggest any limitation as to the scope of use or functionality of the computer software implementing embodiments of the present disclosure. Neither should the configuration of components be interpreted as having any dependency or requirement relating to any one or combination of components illustrated in the exemplary embodiment of a computer system (900).

Computer system (900) may include certain human interface input devices. Such a human interface input device may be responsive to input by one or more human users through, for example, tactile input (such as: keystrokes, swipes, data glove movements), audio input (such as: voice, clapping), visual input (such as: gestures), olfactory input (not depicted). The human interface devices can also be used to capture certain media not necessarily directly related to conscious input by a human, such as audio (such as: speech, music, ambient sound), images (such as: scanned images, photographic images obtain from a still image camera), video (such as two-dimensional video, three-dimensional video including stereoscopic video).

Input human interface devices may include one or more of (only one of each depicted): keyboard (901), mouse (902), trackpad (903), touch screen (910), data-glove (not shown), joystick (905), microphone (906), scanner (907), camera (908).

Computer system (900) may also include certain human interface output devices. Such human interface output devices may be stimulating the senses of one or more human users through, for example, tactile output, sound, light, and smell/taste. Such human interface output devices may include tactile output devices (for example tactile feedback by the touch-screen (910), data-glove (not shown), or joystick (905), but there can also be tactile feedback devices that do not serve as input devices), audio output devices (such as: speakers (909), headphones (not depicted)), visual output devices (such as screens (910) to include CRT screens, LCD screens, plasma screens, OLED screens, each with or without touch-screen input capability, each with or without tactile feedback capability—some of which may be capable to output two dimensional visual output or more than three dimensional output through means such as stereographic output; virtual-reality glasses (not depicted), holographic displays and smoke tanks (not depicted)), and printers (not depicted).

Computer system (900) can also include human accessible storage devices and their associated media such as optical media including CD/DVD ROM/RW (920) with CD/DVD or the like media (921), thumb-drive (922), removable hard drive or solid-state drive (923), legacy magnetic media such as tape and floppy disc (not depicted), specialized ROM/ASIC/PLD based devices such as security dongles (not depicted), and the like.

Those skilled in the art should also understand that term “computer readable media” as used in connection with the presently disclosed subject matter does not encompass transmission media, carrier waves, or other transitory signals.

Computer system (900) can also include an interface (954) to one or more communication networks (955). Networks can for example be wireless, wireline, optical. Networks can further be local, wide-area, metropolitan, vehicular and industrial, real-time, delay-tolerant, and so on. Examples of networks include local area networks such as Ethernet, wireless LANs, cellular networks to include GSM, 3G, 4G, 5G, LTE and the like, TV wireline or wireless wide area digital networks to include cable TV, satellite TV, and

terrestrial broadcast TV, vehicular and industrial to include CANBus, and so forth. Certain networks commonly require external network interface adapters that attached to certain general purpose data ports or peripheral buses (949) (such as, for example USB ports of the computer system (900)); others are commonly integrated into the core of the computer system (900) by attachment to a system bus as described below (for example Ethernet interface into a PC computer system or cellular network interface into a smart-phone computer system). Using any of these networks, computer system (900) can communicate with other entities. Such communication can be uni-directional, receive only (for example, broadcast TV), uni-directional send-only (for example CANbus to certain CANbus devices), or bi-directional, for example to other computer systems using local or wide area digital networks. Certain protocols and protocol stacks can be used on each of those networks and network interfaces as described above.

Aforementioned human interface devices, human-accessible storage devices, and network interfaces can be attached to a core (940) of the computer system (900).

The core (940) can include one or more Central Processing Units (CPU) (941), Graphics Processing Units (GPU) (942), specialized programmable processing units in the form of Field Programmable Gate Areas (FPGA) (943), hardware accelerators for certain tasks (944), graphics adapters (950), and so forth. These devices, along with Read-only memory (ROM) (945), Random-access memory (946), internal mass storage such as internal non-user accessible hard drives, SSDs, and the like (947), may be connected through a system bus (948). In some computer systems, the system bus (948) can be accessible in the form of one or more physical plugs to enable extensions by additional CPUs, GPU, and the like. The peripheral devices can be attached either directly to the core's system bus (948), or through a peripheral bus (949). In an example, the screen (910) can be connected to the graphics adapter (950). Architectures for a peripheral bus include PCI, USB, and the like.

CPUs (941), GPUs (942), FPGAs (943), and accelerators (944) can execute certain instructions that, in combination, can make up the aforementioned computer code. That computer code can be stored in ROM (945) or RAM (946). Transitional data can also be stored in RAM (946), whereas permanent data can be stored for example, in the internal mass storage (947). Fast storage and retrieve to any of the memory devices can be enabled through the use of cache memory, that can be closely associated with one or more CPU (941), GPU (942), mass storage (947), ROM (945), RAM (946), and the like.

The computer readable media can have computer code thereon for performing various computer-implemented operations. The media and computer code can be those specially designed and constructed for the purposes of the present disclosure, or they can be of the kind well known and available to those having skill in the computer software arts.

As an example and not by way of limitation, the computer system having architecture (900), and specifically the core (940) can provide functionality as a result of processor(s) (including CPUs, GPUs, FPGA, accelerators, and the like) executing software embodied in one or more tangible, computer-readable media. Such computer-readable media can be media associated with user-accessible mass storage as introduced above, as well as certain storage of the core (940) that are of non-transitory nature, such as core-internal mass storage (947) or ROM (945). The software implementing various embodiments of the present disclosure can be

stored in such devices and executed by core (940). A computer-readable medium can include one or more memory devices or chips, according to particular needs. The software can cause the core (940) and specifically the processors therein (including CPU, GPU, FPGA, and the like) to execute particular processes or particular parts of particular processes described herein, including defining data structures stored in RAM (946) and modifying such data structures according to the processes defined by the software. In addition or as an alternative, the computer system can provide functionality as a result of logic hard-wired or otherwise embodied in a circuit (for example: accelerator (944)), which can operate in place of or together with software to execute particular processes or particular parts of particular processes described herein. Reference to software can encompass logic, and vice versa, where appropriate. Reference to a computer-readable media can encompass a circuit (such as an integrated circuit (IC)) storing software for execution, a circuit embodying logic for execution, or both, where appropriate. The present disclosure encompasses any suitable combination of hardware and software.

While this disclosure has described several exemplary embodiments, there are alterations, permutations, and various substitute equivalents, which fall within the scope of the disclosure. It will thus be appreciated that those skilled in the art will be able to devise numerous systems and methods which, although not explicitly shown or described herein, embody the principles of the disclosure and are thus within the spirit and scope thereof.

What is claimed is:

1. A method of mesh processing performed in a video encoder, the method comprising:

determining a plurality of neighboring vertices of a current vertex in a current frame of a mesh in one of a two-dimensional model and a three-dimensional model, the plurality of neighboring vertices and the current vertex being included in the current frame and corresponding to the mesh at a first time instance, each of the plurality of neighboring vertices being connected to the current vertex through a respective edge in the mesh;

determining a plurality of neighboring estimation errors of the plurality of neighboring vertices of the current vertex, each of the plurality of neighboring estimation errors indicating a difference between a reference vertex of a corresponding one of the plurality of neighboring vertices in a reference frame of the mesh and the corresponding one of the plurality of neighboring vertices in the current frame at the first time instance, the reference frame corresponding to the mesh at a second time instance that is different from the first time instance;

when the plurality of neighboring estimation errors of the plurality of neighboring vertices is determined, determining a prediction residue of the current vertex based on the plurality of neighboring estimation errors of the plurality of neighboring vertices; and generating prediction information of the current vertex based on the determined prediction residue of the current vertex.

2. The method of claim 1, wherein the reference vertex of the corresponding one of the plurality of neighboring vertices is positioned at a same relative position in the reference frame as the current vertex in the current frame, the reference frame and the current frame being generated at different time instances.

23

3. The method of claim 2, further comprising:
determining an average neighboring estimation error of
the plurality of neighboring estimation errors,
wherein the determining the prediction residue includes
determining the prediction residue of the current vertex
based on the average neighboring estimation error. 5
4. The method of claim 3, wherein the determining the
prediction residue of the current vertex further comprises:
determining an estimation error of the current vertex
based on a difference between a reference vertex of the
current vertex and the current vertex; and 10
determining a prediction list of the current vertex, pre-
dictors of the prediction list including the estimation
error of the current vertex, the average neighboring
estimation error subsequent to the estimation error of
the current vertex, and the plurality of neighboring
estimation errors subsequent to the average neighbor-
ing estimation error, a respective prediction index being
associated with each of the predictors in the prediction
list. 20
5. The method of claim 4, wherein the plurality of
neighboring vertices is ordered based on an Edgebreaker
algorithm in which the plurality of neighboring vertices is
traversed in a spiraling triangle-spanning-tree order. 25
6. The method of claim 4, wherein the determining the
prediction residue of the current vertex further comprises:
determining (i) a difference between the average neigh-
boring estimation error and the estimation error of the
current vertex, and (ii) a difference between each of the
plurality of neighboring estimation errors and the esti-
mation error of the current vertex; 30
selecting a neighboring estimation error from the average
neighboring estimation error and the plurality of neigh-
boring estimation errors, the selected neighboring esti-
mation error having a minimum difference; and 35
determining the prediction residue of the current vertex as
one of (i) the estimation error of the current vertex and
(ii) the selected neighboring estimation error.
7. The method of claim 6, wherein the determining the
prediction residue of the current vertex further comprises:
determining a prediction error based on one of the plu-
rality of neighboring vertices of the current vertex
according to an inter frame prediction; 40
comparing the prediction error with each of the predictors
in the prediction list; and 45
determining the prediction residue as the prediction error
when the prediction error is smaller than the predictors
in the prediction list.
8. The method of claim 7, wherein the determining the
prediction residue of the current vertex further comprises:
determining the prediction residue as a mixed estimation
error, the mixed estimation error including one of (i) an
average of the selected neighboring estimation error
and the prediction error and (ii) an average of the
estimation error of the current vertex and the prediction
error. 55
9. The method of claim 8, wherein the prediction infor-
mation of the current vertex further comprises:
a flag that indicates the prediction residue is one of (i) the
estimation error of the current vertex or the selected
neighboring estimation error based on a first value of
the flag, (ii) the prediction error based on a second
value of the flag, and (iii) the mixed estimation error
based on a third value of the flag. 60
10. The method of claim 9, wherein the prediction infor-
mation of the current vertex further comprises: 65

24

- index information indicating which one of the predictors
in the prediction list is the prediction residue based on
the flag being the first value, and
prediction residue information indicating:
(i) when the flag is the first value, the estimation error
of the current vertex when the index information
indicates that the prediction residue is the estimation
error of the current vertex, and the selected neigh-
boring estimation error when the index information
that the prediction residue is the selected neighboring
estimation error, and
(ii) when the flag is the second value, a difference
between the current vertex and a prediction vertex is
indicated by the prediction error, and
(iii) when the flag is the third value, a difference
between the current vertex and a prediction vertex is
indicated by the mixed estimation error.
11. The method of claim 7, wherein the inter frame
prediction further comprises one of:
a delta prediction that is based on a difference between a
first prediction vertex and the current vertex, the first
prediction vertex including the one of the plurality of
neighboring vertices of the current vertex, and
a parallelogram prediction in which a second prediction
vertex is determined based on a prediction triangle of a
plurality of triangles of the mesh, the prediction tri-
angle sharing an edge with a triangle of the plurality of
triangles that includes the current vertex, the second
prediction vertex being opposite to the shared edge, the
second prediction vertex and the prediction triangle
forming a parallelogram.
12. A method of mesh processing performed in a video
decoder, the method comprising:
receiving a bitstream including coded information of a
mesh in a current frame;
determining a plurality of neighboring vertices of a cur-
rent vertex in the mesh in one of a two-dimensional
model and a three-dimensional model, the plurality of
neighboring vertices and the current vertex being
included in the current frame and corresponding to the
mesh at a first time instance, each of the plurality of
neighboring vertices being connected to the current
vertex through a respective edge in the mesh;
determining a plurality of neighboring estimation errors
of the plurality of neighboring vertices of the current
vertex, each of the plurality of neighboring estimation
errors indicating a difference between a reference ver-
tex of a corresponding one of the plurality of neigh-
boring vertices in a reference frame of the mesh and the
corresponding one of the plurality of neighboring ver-
tices in the current frame at the first time instance, the
reference frame corresponding to the mesh at a second
time instance that is different from the first time
instance;
when the plurality of neighboring estimation errors of the
plurality of neighboring vertices is determined, deter-
mining a prediction residue of the current vertex based
on the plurality of neighboring estimation errors of the
plurality of neighboring vertices; and
reconstructing the current vertex based on the determined
prediction residue of the current vertex.
13. The method of claim 12, wherein the reference vertex
of the corresponding one of the plurality of neighboring
vertices is positioned at a same relative position in the
reference frame as the current vertex in the current frame,
the reference frame and the current frame being generated at
different time instances.

25

14. The method of claim 13, further comprising:
determining an average neighboring estimation error of
the plurality of neighboring estimation errors,
wherein the determining the prediction residue includes
determining the prediction residue of the current vertex
based on the average neighboring estimation error.

15. The method of claim 14, wherein the determining the
prediction residue of the current vertex further comprises:
determining an estimation error of the current vertex
based on a difference between a reference vertex of the
current vertex and the current vertex; and
determining a prediction list of the current vertex, pre-
dictors of the prediction list including the estimation
error of the current vertex, the average neighboring
estimation error subsequent to the estimation error of
the current vertex, and the plurality of neighboring
estimation errors subsequent to the average neighbor-
ing estimation error, a respective prediction index being
associated with each of the predictors in the prediction
list.

16. The method of claim 15, wherein the plurality of
neighboring vertices is ordered based on an Edgebreaker
algorithm in which the plurality of neighboring vertices is
traversed in a spiraling triangle-spanning-tree order.

17. The method of claim 15, wherein the determining the
prediction residue of the current vertex further comprises:
determining (i) a difference between the average neigh-
boring estimation error and the estimation error of the
current vertex, and (ii) a difference between each of the
plurality of neighboring estimation errors and the esti-
mation error of the current vertex;
selecting a neighboring estimation error from the average
neighboring estimation error and the plurality of neigh-
boring estimation errors, the selected neighboring esti-
mation error having a minimum difference; and
determining the prediction residue of the current vertex as
one of (i) the estimation error of the current vertex and
(ii) the selected neighboring estimation error.

18. The method of claim 17, wherein the determining the
prediction residue of the current vertex further comprises:
determining a prediction error based on one of the plu-
rality of neighboring vertices of the current vertex
according to an inter frame prediction;
comparing the prediction error with each of the predictors
in the prediction list; and

26

determining the prediction residue as the prediction error
when the prediction error is smaller than the predictors
in the prediction list.

19. The method of claim 18, wherein the determining the
prediction residue of the current vertex further comprises:
determining the prediction residue as a mixed estimation
error, the mixed estimation error including one of (i) an
average of the selected neighboring estimation error
and the prediction error and (ii) an average of the
estimation error of the current vertex and the prediction
error.

20. A method of processing mesh data, the method
comprising:
processing a bitstream of the mesh data according to a
format rule, wherein:
the bitstream includes coded information of a mesh in a
current frame; and
the format rule specifies that:

a plurality of neighboring vertices is determined for a
current vertex in the mesh in one of a two-dimen-
sional model and a three-dimensional model, the
plurality of neighboring vertices and the current
vertex being included in the current frame and cor-
responding to the mesh at a first time instance, each
of the plurality of neighboring vertices being con-
nected to the current vertex through a respective
edge in the mesh;

a plurality of neighboring estimation errors of the
plurality of neighboring vertices of the current vertex
is determined, each of the plurality of neighboring
estimation errors indicating a difference between a
reference vertex of a corresponding one of the plu-
rality of neighboring vertices in a reference frame of
the mesh and the corresponding one of the plurality
of neighboring vertices in the current frame at the
first time instance, the reference frame correspond-
ing to the mesh at a second time instance that is
different from the first time instance;

when the plurality of neighboring estimation errors of
the plurality of neighboring vertices is determined, a
prediction residue of the current vertex is determined
based on the plurality of neighboring estimation
errors of the plurality of neighboring vertices; and
the current vertex is processed based on the determined
prediction residue of the current vertex.

* * * * *